



浙江财经大学

本科生毕业论文(设计)

题目：基于 GAN 和 FLOW 模型的仿声系统的研究与实现

学生姓名：	许骏亮
学 号：	200110730225
指导教师：	张帅
所在学院：	信息技术与人工智能学院
专业名称：	软件工程
班 级：	20 软件 1 班

2024 年 5 月

声明及论文使用的授权

本人郑重声明所呈交的论文是我个人在导师的指导下独立完成的。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写的研究成果。

论文作者签名：

年 月 日

本人同意浙江财经大学有关保留使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以上网公布全部内容，可以采用影印、缩印或其他复制手段保存论文。

论文作者签名：

年 月 日

基于 GAN 和 FLOW 模型的仿声系统的研究与实现

摘要：近年来，随着人工智能技术在文字和图像领域的迅猛发展，其在音频领域也开始崭露头角。伴随着语音识别、文字转语音和语音转文字等算法的相继提出，音色的分离和转换也开始成为研究的热点，为人工智能在音频处理领域开辟了一条新的道路。本文尝试以男歌手周杰伦的音色和其他音色的特征转换为例，进行数据的采集和处理，在此基础上构建数据集；基于生成对抗网络 (Generative Adversarial Network) 算法及流动生成模型 (FLOW-based model) 的计算思路，设计了一种新的音色转换算法，旨在高速度高性能地提取音色的梅尔倒谱特征并进行相应的处理和转化，为用户提供一个较为精准和快速的音色转换系统。本文将全方位的分析算法的可行性和优越性，最终在 Web 网页端实现音色的转换功能，并呈现出音频数据的波形图和梅尔频谱图等可视化特征。

关键词：音色转换；GAN；FLOW；可视化分析

Research and implementation of sound imitation system based on GAN and FLOW models

Abstract: In recent years, with the rapid development of artificial intelligence technology in the field of text and images, it has also begun to emerge in the field of audio. With the successive proposals of algorithms such as speech recognition, text-to-speech, and speech-to-text, timbre separation and conversion have also begun to become a research hotspot, opening up a new path for artificial intelligence in audio processing. This article attempts to use the feature conversion of male singer Jay Chou's timbre and other timbres as an example to collect and process data, and build a data set on this basis; based on the Generative Adversarial Network algorithm and the calculation idea of the Flow-based generation model, designed a new timbre conversion algorithm, aiming to extract the Mel-Spectrum characteristics of timbre with high speed and high performance and perform corresponding processing and transformation, providing users with a more accurate and fast

timbre conversion system. This article will comprehensively analyze the feasibility and superiority of the algorithm, and finally implement the timbre conversion function on the Web page, and present visual features such as waveform diagrams and Mel spectrograms of audio data.

Key words: Timbre conversion; GAN; FLOW; Visual analysis

目 录

摘 要	I
Abstract	I
1 引 言	1
1.1 研究背景及意义	1
1.2 论文组织架构	2
2 相关工作	3
2.1 音色转换算法模型	3
2.2 生成模型	4
3 数据处理和任务概要	5
3.1 数据处理	5
3.1.1 数据概况	5
3.1.2 数据集的构建	5
3.2 任务概要	7
3.2.1 算法任务概要	7
3.2.2 前后端设计概要	8
4 基于 GAN 和 FLOW 模型的仿声系统	9
4.1 FLOW 模型和损失函数的构建	9
4.1.1 FLOW 模型简介	9
4.1.2 损失函数的构建	11
4.2 模型和搭建策略	12
4.2.1 门控线性单元和特征放大策略	13
4.2.2 对抗训练和流动计算结合策略	14
4.2.3 L2 正则化和残差策略	15
4.3 训练策略	16
4.3.1 学习率递减策略	16
4.3.2 梯度调整策略	17
4.4 模型创新点	18
5 评价指标和实验结果分析	20
5.1 评价指标	20
5.1.1 主观指标	20
5.1.2 客观指标	20

5.2	实验结果分析	21
5.2.1	实验环境设置	21
5.2.2	模型评价	21
6	Web 网站前后端设计	25
6.1	系统主页面	25
6.2	文件上传模块	26
6.3	音频数据特征展示模块	27
6.4	音频特征可视化模块	29
6.5	音色转换模块	31
7	总结	33
	参考文献	34
	致 谢	36

1 引言

1.1 研究背景及意义

近年来，音频处理技术愈发受到关注，人工智能领域对音频处理的技术需求越来越大，主要包括了音频识别、文字换音频、AI 音乐创作、音频增强以及音色转换等领域的技术，通过对技术算法的研究，不但充分缓解了相关领域工作人群的压力，也极大丰富了人们的生活。

音色的多样性是音乐和语言交流中一个非常关键的元素。它不仅能够反映个体的声音特征，还能传递情感和信息。每个人的声音都拥有独一无二的音色，这种音色会随着人的情绪、健康状况、年龄甚至环境的改变而发生微妙的变化。然而，在日常生活中，人们往往维持着一种相对稳定的音色来进行交流和沟通。在音色转换算法的研究领域，学者们已经取得了显著的进展。Kim^[1]等人在 2021 年提出了一种创新的音频处理算法 VITS，该算法结合了文字转音频算法 TTS、数据增强算法 VAE^[2]和流动模型 FLOW^[3]，形成了一个强大的音频处理框架，VITS 在语音识别和音色转换方面取得了突破性的成果，尤其是在进行非标准发音和方言音色转换时表现出色；同年，郝一亚^[4]等人利用梅尔频谱和频谱通量设计出了一种新的卷积网络，这种网络能够实时检测和识别音乐，并且能够有效地分离目标音乐和背景噪声，这项技术的应用，不仅提高了音乐识别的精度，也为音色转换技术的发展提供了新的视角；Sheinin^[5]等人在 2022 年提出的 Dual-Shutter 方法，通过使用慢速照相机捕捉多个场景源下的高速震动表面，并通过分析由目标声音引起的表面震动来实现噪音的消除和声音的分离，这种方法为声音信号处理提供了一种新的物理手段，进一步拓展了音色转换技术的应用场景；2023 年，李涛^[6]等人针对音色转换中跨语种转换效果不理想的问题，提出了 DICLET-TTS 算法。该算法结合了扩散模型和情感迁移算法，有效地提升了跨语种音色转换的效果，为多语言环境下的音色转换提供了新的解决方案。同年，Yang^[7]等人介绍了扩散模型算法 (Diffusion Models) 的方法和应用，使得扩散模型在深度学习的各个领域得到了更好的应用。

随着人工智能技术的不断进步，音色转换技术的研究已经不仅仅局限于音色的转换本身。研究者们开始关注情感的迁移、语音的增强以及人声与背景声的分离等更深层次的问题。这些研究不仅推动了音色转换技术的发展，也为语音识别、语音合成和音乐制作等领域带来了新的机遇。随着技术的不断成熟，音色转换技术将在未来的音乐制作、

语言交流和个人身份验证等方面发挥更加重要的作用。

AI 音色转换技术是一项声音处理技术，它通过深入分析不同音色之间的细微差别，提取出每种音色独有的特征，并进行对比分析训练。这个过程涉及到对声音信号的频谱、节奏、强度、音色等多维度特征的深入理解。通过这种训练，AI 模型在如何在保持情感、音调等其他声音特征不变的情况下，对核心音色特征进行精确的转换。这种技术的应用不局限于音频娱乐领域，它在实时广播、智能字幕生成、隐私保护等多个方面都发挥着重要作用。在实时广播中，AI 音色转换技术可以模拟特定播音员的声音，即使该播音员不在场，也能保持节目的连贯性和个性化风格。在智能字幕生成方面，通过音色转换，可以生成与说话人音色相匹配的字幕，提升观众的观看体验。此外，在隐私保护领域，音色转换技术可以用来生成匿名化的声音，保护个人隐私不被识别。传统的音色转换算法往往依赖于复杂的录音设备和繁琐的处理流程。它们通常采用将背景声和人声分离的方法，在保持背景音不变的情况下进行音色转换。这种方法需要使用各种仪器来录制和保存音色特征，不仅过程繁琐，而且效率低下。近年来，随着深度学习技术的发展，梅尔倒谱特征在音色转换领域受到了越来越多的重视。梅尔倒谱特征是一种能够捕捉到声音音色特征参数，它通过模拟人耳对不同频率声音的感知能力，提取出声音的关键信息。相较于传统方法，基于深度学习的音色转换技术具有显著的优势。它能够智能地分析和学习音色的各种特征，自动保存并转换所需的音色特征，从而更好地满足工业生产的需求。深度学习模型强大的计算能力和自学习能力，使得音色转换过程更加自动化和智能化，极大地提高了音色转换的效率和质量。随着技术的不断进步，未来 AI 音色转换技术有望在更多领域得到应用，为人们的生活带来更多便利。

1.2 论文组织架构

本文主要介绍基于生成对抗网络的音色转换算法模型研究工作的全过程，包括了生成对抗网络中生成器 (Generator) 和判别器 (Discriminator) 对抗训练的思想，结合门控线性单元^[8](GLU)、残差思想 (Resnet)、L2 正则 (L2Norm) 以及流动模型 (FLOW) 等策略，实现音色转换这一目标。本文的其余组织结构如下：第二节将对音频处理领域算法技术的相关工作进行梳理和分析；第三节将对实验中使用的数据进行介绍，并分析实验任务和系统概览；第四节将对本文采用的算法模型、技术策略以及创新点进行阐述；第五节分析算法训练结果和评价指标；第六节说明前后端系统设计的方案和思路，并阐述使用的工具和方法；第七节中总结本文的研究结果，针对不足点提出未来的提升空间。

2 相关工作

2.1 音色转换算法模型

当前音色转换算法模型的实现方法主要分为基于扩散模型^[7](Diffusion) 的算法和基于生成对抗模型^[9](GAN) 的算法。在扩散模型中, 根据训练噪声评估模型效果以及马尔科夫链^[10]的扩散步骤原理, 它定义了一个新的扩散步骤, 即在扩散过程中缓慢并有序地加入随机高斯噪声, 并在反向传播中恢复得到干净的样本, 在生成模型的辅助下, 可以达到很好的去噪效果, 但是在生成模型在过程中需要不断的生成样本数据, 很容易导致模型的特征冗余。扩散模型中包含了一种加速采样器, 能够允许它生成并处理高维度的变量, 在很多复杂的环境下都适用, 但是同时使得它需要迭代多次, 推理速度明显变慢。

生成对抗网络 (GAN) 是一种深度学习模型, 它通过训练两个相互竞争的网络, 即生成器和判别器, 以生成新的、与真实数据相似的样本。生成器的目标是产生尽可能接近真实数据的样本, 而判别器的任务则是区分真实数据和生成器生成的样本。这种对抗过程促使生成器不断学习和改进, 直至生成的样本足以欺骗判别器, 使其无法准确区分真伪。

与扩散模型不同, 对抗生成网络的生成器首先创建一批样本数据, 然后判别器对这些数据进行评估, 给出一个反馈信号。生成器根据这个反馈信号调整自己的生成策略, 在下一轮生成更高质量的样本。这个过程不断重复, 直到判别器无法有效地区分生成的数据和真实数据, 此时生成器生成的样本在统计上与真实数据无异, 模型的训练达到稳定状态。生成对抗网络在处理特定类型的数据, 如图像、音频等, 尤其在需要捕捉和生成复杂特征时表现出色。2020 年, Kaneko^[11]等人提出了一种非并行语音转换算法 CycleGAN-VC3, 该算法利用时频自适应归一化 (TFAN) 技术来调整和转换数据特征的比例和偏差, 通过这种方式, 算法能够更好地捕捉和反映原数据的梅尔频谱图的时频结构特征, 从而在音色转换任务中取得了显著的效果。同年, Choi^[12]等人提出了 StarGAN-VC2 算法, 这是一种支持多目标特征转换的算法, 通过构建一个公用的生成器, 并采用星型结构设计模型框架, 使得任意两个特征之间都可以进行转换, 此外, 该算法在损失函数中引入了风格损失函数, 专注于特征风格本身, 以更好地保护特征的完整性, 这对于生成具有特定风格的语音样本具有重要意义。2022 年, 雷怡^[13]等人提出了 Glow-WaveGan 2 算法, 这是一种利用一句话来生成模型从未训练过的语音的算法, 该

算法使用了与 VITS^[1]相同的变分自编码器 VAE^[2]，并结合了源数据编码器生成的零资源语音 (zero-shot)。这种方法能够生成高质量的新说话人语音，对于音频数据增强领域具有重要的启发意义，能够有效地扩充样本数据，提高数据的多样性和丰富性。

通过生成对抗网络算法训练的模型虽然可以更快的推理和拟合，但是在高维度大数据场景下更需要扩散模型的效果，并且当数据质量欠佳的情况下，可能会导致生成对抗模型的性能和准确率下降，同时判别器也会因为特征的不稳定难以判别生成样本的真伪。此外，生成对抗网络的训练不稳定，通常需要更多的人为手动干预，才能确保训练的高效进行，所以精确高效的音色转换在该领域仍是一个重大难题。

2.2 生成模型

目前主流的生成模型包括了自回归模型^[14] (Autoregressive Models)、变分自编码器^[2] (VAE)、生成对抗网络^[9](GAN)、流动生成模型^[3](FLOW) 以及去噪扩散概率模型^[15](DDPM)。Binkowski^[14]等人提出的自回归模型通过将自回归算法与卷积神经网络进行结合，将一维时间序列的多维特征值看作数组进行卷积操作，通过训练以对已有的时间序列进行预测推理，虽然可以做到准确的概率估计，但是训练和计算速度相对较慢；Kingma^[2]等提出的变分自编码器是一种无监督生成模型，通过对已有数据样本的分布来模拟未知样本的分布，通过模型的训练让两者相互近似，以达到样本和目标特征近似的效果，从而实现特征的转换，但是生成的数据样本的质量相较于对抗生成网络欠佳；Dinh^[3]等提出的流动生成模型是一种非常独特的生成模型，不同于其他生成模型采用的对抗训练和分布近似逼近的方法，它选择直接计算概率，将数据的分布转换为积分公式，算出数据样本的分布，计算的可逆性使得这样的策略可以保持特征转换过程的可逆性，极大的缩短了训练所需要的时间，但是在设计上难度较大；Ho^[15]等人提出的去噪扩散概率模型结合了马尔科夫链^[10]和变分自编码器的思路，固定了编码器的层级，它将前向过程作为编码器，将反向过程作为解码器，在多个过程中共享编码器和解码器，复杂的计算过程和严重的特征冗余，使得其计算量巨大，推理速度偏慢。

尽管生成模型在特征生成和转换上已经体现了良好的效果，但是单一的生成模型往往不能同时保证生成样本特征的高质量 and 参数推理计算的高速度。本文设计了一种基于生成对抗网络和流动生成模型的新模型，结合了生成对抗网络生成高质量特征的特点和流动生成模型对参数计算的高速的新模型，旨在保证音色转换过程中样本和目标音色特征的高质量以及模型训练推理的高性能。

3 数据处理和任务概要

3.1 数据处理

3.1.1 数据概况

在本文中，本人采用了一种系统化的方法来收集和处理音频数据，以确保实验的准确性和可靠性。音频数据的来源多样，包括电视节目、采访以及原声录音等，这些数据为实验提供了丰富的音色特征和语音内容。源说话人的数据选取自著名艺人周杰伦的采访录音。为了保证实验的一致性和可比较性，将原始采访音频进行了精细的切割处理，最终划分成了 149 条独立的数据片段。每条数据片段的时间长度控制在 3 到 8 秒之间，这个时间段既能够包含足够的语音信息，又不至于过长，便于后续的处理和分析。目标音色的数据则来源于原声录音，与源说话人的数据处理方式类似，将这些原声录音同样切割成了 115 条数据片段，每条数据片段的时间长度也在 3 到 8 秒之间。

在实验过程中，除了上述的核心数据集外，本人还广泛收集了其他多种音频数据，并进行了一系列的尝试和测试。这些额外的音频数据为模型提升了选择性和灵活性，根据实验结果，选择最合适的数据集进行进一步的分析和研究。所有收集到的音频数据及其特征都进行了详细的记录和整理，如表 3.1 所示。这些记录包括每条数据的基本属性，如时长、采样率和声道数等。

表 3.1 数据及其特征概况

数据名称	采样率/赫兹	时长/秒	比特率/kps	编码方式	通道数/个	数量/条
xjl	48000	3-8	1300-1500	PCM	1	136
crz	44100	3-8	600-800	PCM	2	115
jay	44100	3-8	600-800	PCM	2	149
anshitou	16000	3-8	600-800	PCM	1	109
xiuyi	16000	3-8	600-800	PCM	1	109
gaoxiaosong	16000	3-8	200-500	PCM	1	111

3.1.2 数据集的构建

数据的处理流程主要包括了一维时间按序列到二维浮点数组的转化、特征的提取以及 Pytorch 的封装等，具体流程见图 3.1 所示。

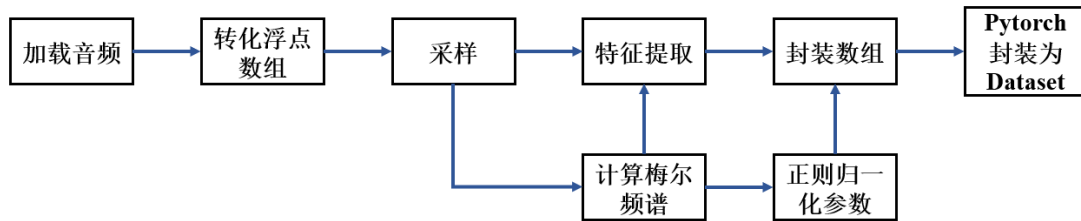


图 3.1 数据集的构建

其核心操作具体步骤如下：

(1) 类型转换和采样特征

将原始音频数据转换为适合机器学习模型处理的格式是至关重要的一步。

首先，将音频数据转换成浮点类型的数组。因为浮点数能够以更高的精度表示音频信号的波形，能够更准确地捕捉声音的细节，并在转换中将音频文件的采样值从整形格式转换为浮点格式。

接着，通过采样操作提取音频数据的基频特征，即音高或声音的基频（F0）。基频是音色转换中一个非常重要的特征，因为它直接影响到声音的音色特征。提取基频通常涉及到对音频信号进行变换，然后识别出频率分量中对应基频的峰值。除了基频特征，音频数据的梅尔频谱系数也是音色转换中十分重要的特征。这些系数能够捕捉声音的音色和纹理，对于音色转换来说十分关键。

完成上述任务的同时，对音频数据的特征进行综合检测，以识别和筛选出不完整或错误的数据条目。这一步骤确保了用于训练和音色转换的数据质量，排除了可能影响模型性能的噪声和异常值。最终，所有提取和计算得到的特征会被封装到一个数组中，形成一种结构化的数据格式，这种格式可以直接用于机器学习模型的训练过程。这个数组包含了音频数据的关键音色特征，为音色转换模型提供了必要的输入信息，使得模型能够学习到不同音色之间的差异，并实现从一种音色到另一种音色的转换。

(2) 数组封装和数据集的整合

在音频特征被成功封装成数组之后，为了提升数据处理的效率和模型训练的效果，接下来的步骤是对这些特征进行一系列的预处理操作。

首先，对音频数据的特征的进行正则化处理。正则化是一种能够高效的减少数据中噪声和异常值影响的方法，它可以提高模型的泛化能力。它可以让数据服从的数据特征

分布更加顺序均匀。这有助于模型更准确地学习各个特征间的关系，避免因特征的尺度不同而引起的推理的困难。

接着，对特征进行对数转换。对数转换可以实现特征的缩放，这一转换有助于模型更好地捕捉到音频信号中的动态范围变化。在对数转换之后，执行归一化操作。归一化将特征规整到 0-1 之间，使得所有特征的尺度保持一致，便于后续的计算和评估。归一化处理可以加快模型的收敛速度，同时减少不同特征尺度带来的影响，提高模型的稳定性和预测性能。

完成上述对音频数据特征的处理后，使用 Pytorch 框架来封装和保存数据集。将处理好的音频数据的特征封装成一个统一的数据集对象。这个对象支持以批量的形式向模型提供数据。通过这一系列的数据处理和封装步骤，为音色转换模型的训练准备了结构化、规范化且高效的输入数据，有效地提高了模型的精度。

3.2 任务概要

在整个设计过程中主要包括两大类任务，即算法的搭建、训练和测试与前后端系统的设计与美化。

3.2.1 算法任务概要

在音色转换任务中，为了实现高效且精确的音色转换，本文采用了一种融合了对抗生成网络 (GAN) 和流动生成模型 (FLOW) 的新模型算法。这种算法结合了两强大的生成模型的优势，旨在生成高质量且逼真的音频数据样本。在算法设计的过程中，首先基于对抗生成网络的核心原理，即生成器和判别器之间的对抗训练，来构建模型的主体结构。生成器负责生成音频样本，而判别器则评估这些样本的真实性，通过这种对抗过程，生成器不断学习并改进其生成的样本质量。为了更精确地计算模型的损失和参数分布，采用流动生成模型的参数计算方法。这种方法使得模型以一种可逆且概率的方式生成样本，从而提供了一种更加灵活和有效的训练策略。

为了保证模型训练的快速和精确，模型中加入了多种策略。引入门控线性单元^[8](GLU)，作为一种参数的门控来筛选重要的特征。门控线性单元制能够帮助模型在推理的过程中，保持时间序列的时序位置来提取特征，从而提高音色转换的准确性，同时并行的结构设计使得参数的运算更加迅速。此外，线性门控单元也被集成到卷积层中，进一步提升了模型对特征的处理能力。

为了解决在音色转换任务中音频特征容易产生冗余的问题，在模型中引入了标准的

残差单元^[16](Res-Block)，以防止在模型推理过程中由于参数冗余导致的过拟合现象。此外，采用了梯度递减策略^[17]和学习率调整策略优化模型的训练过程，梯度递减策略有助于在训练初期快速收敛，而随着训练的进行，逐渐减缓梯度的下降速度，以实现更精细的优化。学习率调整策略则根据模型的训练进度动态调整学习率的大小，以平衡训练的稳定性和速度。

3.2.2 前后端设计概要

为了给用户提供一个流畅且便捷的交互体验，本文在前端设计上选择了 Vue 3 框架结合 Quasar 框架。Quasar 提供了一套丰富的 UI 组件和主题。通过结合 Vue 3 和 Quasar，可以打造出既美观又实用的用户界面，提升用户的操作体验。在后端设计上，选用了基于 Python 的 Fastapi 框架。Fastapi 提供了简洁的语法和强大的功能，非常适合构建高性能的后端服务。在前后端的数据传输方面，采用了 axios 工具，它支持所有现代浏览器，泛用性很强。

在前后端的 Web 系统中，主要需要实现的功能任务有：实现音频文件的上传功能并在 Web 页面的播放预览功能；实现计算音频数据的多方位特征并在页面中展现的功能；实现对音频数据进行音色转换并可以将转换后的音色播放预览的功能；实现对音频数据特征可视化的功能等。

4 基于 GAN 和 FLOW 模型的仿声系统

4.1 FLOW 模型和损失函数的构建

4.1.1 FLOW 模型简介

流动生成模型 (FLOW) 的设计目标是将两种特征的转换方程计算出来。已知待转换特征的分布 $T(t)$ 和目标特征的分布 $G(z)$ ，流动模型致力于找到一个映射关系函数 F ，使得 $T(t)$ 在经过 F 的作用后变为 $G(z)$ ，即 $F:T(t) \rightarrow G(z)$ 。因此，映射函数 F 便是流动模型计算的核心和目标。以二维分布为例，现假设存在一个映射函数 F ，使得 $F:T(t) \rightarrow G(z)$ ，由于分布 $T(t)$ 与分布 $G(z)$ 可以通过 F 进行转化，根据概率分布与积分的关系，在同一空间中，分布 $T(t)$ 与分布 $G(z)$ 的区域有相同的体积，记 $T(t)$ 的变量 t 在 x 方向偏移量为 Δt_x ，在 y 方向上偏移量为 Δt_y ， $G(z)$ 的变量 z 在各个方向的偏移量分别为 Δz_{xx} 、 Δz_{xy} 、 Δz_{yx} 和 Δz_{yy} ，其中 x 、 y 分别表示横方向和总方向，则有体积等量关系，如公式 4-1 所示：

$$G(z') \left| \det \begin{bmatrix} \Delta z_{xx} & \Delta z_{yx} \\ \Delta z_{xy} & \Delta z_{yy} \end{bmatrix} \right| = T(t') \Delta t_x \Delta t_y \quad (4-1)$$

其中 $\det \begin{bmatrix} \Delta z_{xx} & \Delta z_{yx} \\ \Delta z_{xy} & \Delta z_{yy} \end{bmatrix}$ 表示行列式计算，它的计算结果表示分布 G 在平面上的投影面积，对公式 4-1 进行移项可以得到公式 4-2：

$$G(z') \left| \frac{1}{\Delta t_x \Delta t_y} \det \begin{bmatrix} \Delta z_{xx} & \Delta z_{yx} \\ \Delta z_{xy} & \Delta z_{yy} \end{bmatrix} \right| = T(t') \quad (4-2)$$

根据矩阵的性质，可以将 $\frac{1}{\Delta t_x \Delta t_y}$ 带入到矩阵中进行计算，随着 Δt_x 和 Δt_y 的不断减小，可以在矩阵内部进行偏导数运算，通过计算可以得到公式 4-3：

$$G(z') \left| \det(J_F) \right| = T(t') \quad (4-3)$$

其中 J_F 表示 F 映射函数的雅可比行列式，如公式 4-4 所示：

$$J_F = \begin{bmatrix} \frac{\delta z_x}{\delta t_x} & \frac{\delta z_x}{\delta t_y} \\ \frac{\delta z_y}{\delta t_x} & \frac{\delta z_y}{\delta t_y} \end{bmatrix} \quad (4-4)$$

根据矩阵的可逆性，可以通过求逆操作计算出变换矩阵的逆矩阵，计算过程如公式 4-5 所示：

$$G(z') = T(t') \left| \det(J_{F^{-1}}) \right| = T(t') \quad (4-5)$$

其中 F^{-1} 表示映射函数 F 的逆运算，可以得到一个分布的逆转换 $F^{-1}: G(z) \rightarrow T(t)$ 。于是利用流动生成模型的思想，便将分布 T 到分布 G 的映射函数 F 求解出来，同时它也是可逆的，同时也得出了分布 G 到分布 T 的映射函数 F^{-1} 。

流动生成模型将复杂的数据分布通过一系列可逆的、结构化的变换转换为简单分布，与传统的生成模型例如 GAN、VAE 等有着本质的不同。它将分布的转换问题转化为一系列可计算的积分计算问题。通过精心设计的变换流程，模型能够逐步地将数据从一个分布转换到另一个分布，每一步变换都对应一个确定的映射函数。这些映射函数不仅能够将数据从先验分布转换到目标分布，而且由于它们的可逆性，也能够反向地将数据从目标分布转换回先验分布。这种双向转换的能力为模型带来了显著的优势。首先，它允许模型在推理训练过程中实现双向转化，这意味着模型可以同时学习从数据特征到音色特征，以及从音色特征到数据特征的转换，而无需进行两次独立的训练。这种双向学习机制提高了模型的灵活性和效率，使得模型能够更全面地理解和模拟数据分布。其次，流动生成模型采用了直接计算的方法，它不需要像近似逼近方法那样考虑近似的效果。这不仅简化了模型的设计和训练过程，而且也提高了模型的推理速度。在近似方法中，为了提高逼近的准确性，可能需要增加模型的复杂度或采用更精细的逼近策略，这些都可能导致推理速度的下降。而流动生成模型通过可逆变换直接计算映射函数，避免了这些额外的计算开销，从而实现了更快的推理速度。

此外，流动生成模型的可逆性还带来了其他一些优势。例如，因为可以直接从简单分布中采样，它使得模型的采样变得更加方便，然后通过变换流程生成目标分布的样本。同时，可逆过程中的反向变换提供了校正错误的手段，有助于提高模型的稳定性。

4.1.2 损失函数的构建

在模型的损失函数的构建中，采用了均方损失函数 (MSELoss) 和基于 FLOW 模型计算方法的自定义损失函数相结合的策略。

均方损失函数用于对比两类特征的相似程度，均方损失越小表示特征的转换越相近。其计算方式如公式 4-6 所示：

$$loss(x_i, y_i) = (x_i - y_i)^2 \quad (4-6)$$

自定义损失函数包括生成器的损失和判别器的损失，即每次转换时通过计算得出特征之间的对应关系，转换后利用判别器进行判定，给出转换后的特征和目标特征的差距，计算方法沿用了 FLOW 模型的思路。假设现有生成器 G 和判别器 D 对特征 A 和 B 进行转换，首先将特征 A 与特征 B 都送入生成器 G 中，生成出 B 的假特征 $fake_B$ 和 A 的假特征 $fake_A$ ，再将 $fake_B$ 和 $fake_A$ 经由生成器 G 进行第一轮的对抗训练，记作 $First_A$ 和 $First_B$ 。则可以计算出这一轮特征转换的 EpochLoss，计算过程如公式 4-7 所示：

$$EpochLoss = mean(|A - First_A|) + mean(|B - First_B|) \quad (4-7)$$

其中 mean 表示取均值操作。在理想情况下，特征完全被转化，即生成器生成的特征 A 与目标特征完全相同，记作 $True_A$ ，同理有 $True_B$ 。则在理想情况下的损失 IdealLoss 的计算方法如公式 4-8 所示：

$$IdealLoss = mean(|A - True_A|) + mean(|B - True_B|) \quad (4-8)$$

将 A、B、 $First_A$ 和 $First_B$ 放入判别器 D 中进行判别，得到判别器的反馈特征 D_A 、 D_B 、 D_{fake_A} 和 D_{fake_B} 。于是生成器的损失 GLoss 可以通过均方的方式求出，以特征 A 转换为特征 B 为例，计算方法如公式 4-9 所示：

$$GLoss(A \rightarrow B) = mean(1 - D_{fake_B})^2 \quad (4-9)$$

同理可计算出生成器对于特征 B 转换为特征 A 的损失 $GLoss(B \rightarrow A)$ 。于是根据上述计算结果，通过简单的加权，就可以计算出生成器在这一轮的总损失 GLoss，计算方

法如公式 4-10 所示：

$$GLoss = GLoss(A \rightarrow B) + GLoss(B \rightarrow A) + W_1 * EpochLoss + W_2 * IdealLoss \quad (4-10)$$

其中 W_1 和 W_2 分别表示 EpochLoss 和 IdealLoss 在生成器损失 GLoss 中的权重。同理也可以计算出判别器 D 的损失。通过以上计算得出了在特征转换过程中的生成器的损失 GLoss，通常情况下 GLoss 在训练中表现的越小，表示着生成器生成的样本数据的特征与目标数据特征更为接近。

4.2 模型和搭建策略

为了提高模型拟合和推理计算的速度同时确保模型不会出现过拟合的问题，在模型的结构搭建中，加入了门控线性单元^[8]、残差单元、L2 正则化策略、特征放大策略^[18]和流动模型计算推理思想等方法。如图 4.1 所示，在模型结构中，包括了卷积层、下采样层和残差层以及多个子层。每一层都设计了正则化单元防止网络参数冗余。残差层包含 6 个残差单元保证模型的梯度安全，防止特征冗余导致的梯度爆炸。此外在二维卷积层中还设计了门控线性单元和特征放大单元，以提取到更多的核心特征和过滤冗余特征。

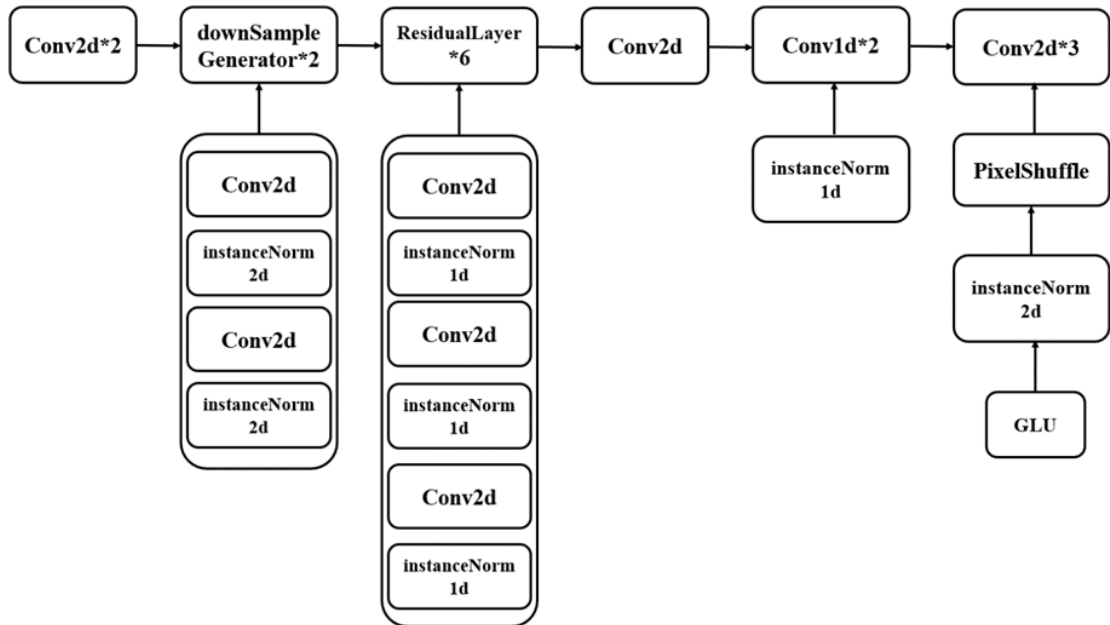


图 4.1 网络模型结构

4.2.1 门控线性单元和特征放大策略

门控线性单元^[8](Gated Linear Unit, GLU) 是一种时序函数，它在处理序列数据时表现出了显著的优势。GLU 的核心思想是通过引入门控机制来控制信息的流动，允许模型在处理数据时更加灵活和动态。在卷积神经网络 (CNN) 中堆叠 GLU 可以有效地表示长时间序列数据，这是因为 GLU 能够捕捉到序列中不同时间点之间的复杂关系，并提取出更高层次、更抽象的特征表示。

与长短期记忆网络^[19](LSTM) 和循环神经网络^[20](RNN) 模型相比，GLU 在处理时间序列数据时采用了时间窗口策略。这种策略通过将序列数据划分为较小的时间段或窗口，然后在这些窗口内应用 GLU，从而避免了直接在原始长序列上进行处理。这种方法可以显著减少模型的非线性计算量，因为它将序列分解为更易于管理的部分，并且可以更集中地捕捉局部依赖关系。此外，GLU 结合卷积计算，可以在时间窗口内对特征进行压缩和转换，这有助于提取更有意义的特征。卷积层通过其局部感受野和权重共享的特性，能够从时间窗口中学习到局部特征，并将这些局部特征整合成全局特征。由于卷积操作是线性的，它在保证时序信息不变的同时，可以直接处理各个时刻的时间数据，而不需要像 LSTM 或 RNN 那样进行逐步的迭代计算。

如图 4.2 所示，门控线性单元的设计思想是将输入的变量 x ，经由参数为 W 和 b 的卷积层处理，再被参数为 V 和 c 的卷积层处理并激活输出门以控制模型最终的输出。其计算公式如公式 4-11 所示：

$$h_l(x) = (x * W + b) \oplus \sigma(x * V + c) \quad (4-11)$$

其中 h_l 表示网络再输出层中的隐藏层中的信息， σ 为第二层卷积层的权重参数。

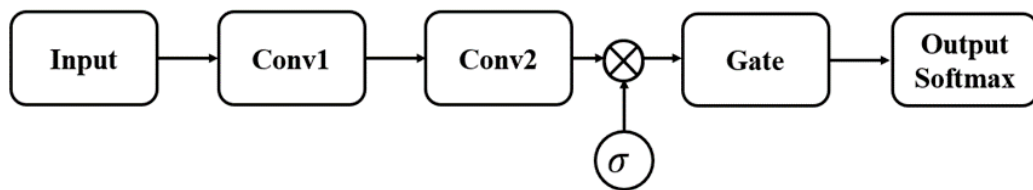


图 4.2 门控线性单元

此外，在线性门控单元前加入了上采样特征放大模块^[18](PixelShuffle)，与传统的通过插值或者解卷积的上采样方法不同，它通过将卷积过程中被窗口化的数据平铺成 $n \times n$

的一个大像素，其中 n 为缩放倍数，以对卷积过程中缩小了的特征图进行有效的放大，使得主要特征能够更好的被提取和训练。假设输入的数据维度为 $(1, 64, 20, 30)$ ，通过特征放大模块放大 2 倍，即 $n=2$ ，可以得到一个维度为 $(1, 16, 40, 60)$ 的输出，可以看到通道数的减少换来了高度和宽度的放大，使得特征更为明显，有效提升了模型提取特征的精准度。结合线性门控单元对特征的深层次捕捉和高性能的计算，能够十分有效的提高模型对于特征的提取和转效能，也很大提升了模型推理计算的速度。

4.2.2 对抗训练和流动计算结合策略

为了解决传统时序特征转换算法中推理速度慢和特征冗余的问题，引入了对抗生成网络^[9](GAN) 来缓解特征的冗余，引入流动生成模型^[3](FLOW) 的计算方法来加速模型在参数推理过程中的速度。在 4.1.2 小节中介绍了利用流动生成模型计算模型损失的过程，相比于传统的分布近似方法，有着很大的速度提升。对抗生成网络采用了对抗训练的方法，不同于扩散模型^[7](Diffusion) 通过对噪声的计算来评价模型，它通过生成器与判别器的对抗训练反馈进行模型的推理。

如图 4.3 所示，生成器 G 将生成的伪样本与真样本一起送入判别器 D 中，判别器 D 判别样本数据的真伪，并给出反馈，如果判别为真则给出 True 反馈，否则反馈伪特征与真特征的差别给生成器 G 。随后生成器根据反馈重新生成数据送入判别器，循环往复直至判别器将生成器生成的数据判断为真。

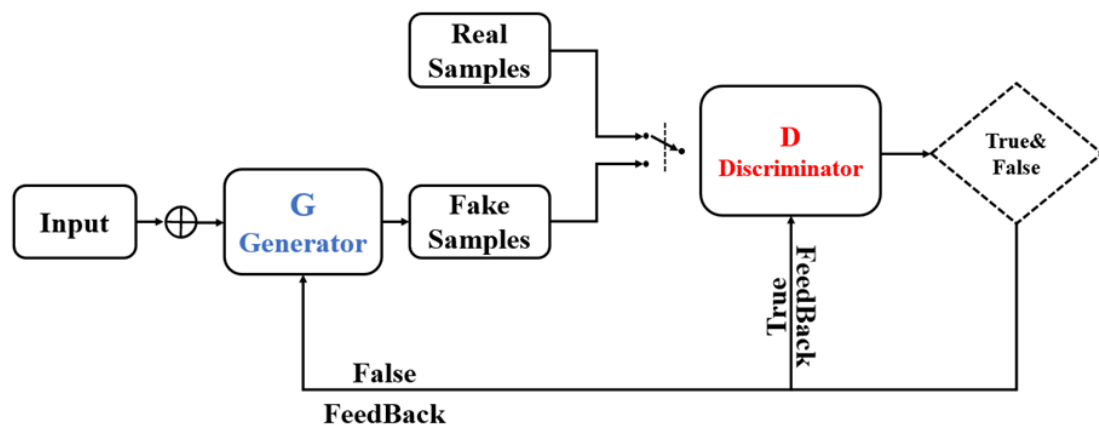


图 4.3 对抗生成网络

通过模型的对抗训练方法，生成器在与判别器两者在对抗过程中不断优化，能够生成更加逼真的样本数据。当判别器对生成器的样本数据的输出给出反馈，生成器会调整其生成策略，保留那些与真实样本数据特征相似的部分，同时将判别器反馈不一致的特

征进行重新生成。在这样的对抗过程中，生成器需要生成的特征会越来越少，从而有效避免了特征冗余的问题。

但是，传统的对抗生成网络 (GAN) 在训练过程中面临着如模型难以收敛、训练困难和梯度不稳定等问题。这些问题往往是由于生成的数据与真实数据的分布存在差异，导致判别器难以提供有效的反馈。为了优化这类问题，引入流动生成模型 (FLOW) 的推理计算思路，通过直接计算分布的方法，避免了传统 GAN 中依赖于近似计算的问题，从而更准确地捕捉数据的分布特性。这种直接计算的方法有助于生成器更好地拟合真实数据的分布，解决了训练过程中的不确定性和梯度不稳定的问题。

4.2.3 L2 正则化和残差策略

在模型的训练和搭建过程中，本文加入了 L2 正则化策略和基于残差网络^[16](Resnet) 的残差单元，来减少模型在推理过程中的参数计算量，以避免模型中出现的过拟合现象。

如公式 4-12 所示，在模型原有的推理计算过程 $L(w)$ 中，添加了惩罚因子 λ 来对每一层的参数 w 进行惩罚。

$$J_{L2}(w) = L(w) + \lambda |w| \quad (4-12)$$

假设 $L(w)$ 在 0 处的导数为 d ，则可以通过求导建立等式，如公式 4-13 所示：

$$\frac{\delta L(2)}{\delta w} \Big|_{w=0} = d \quad (4-13)$$

加入 L2 正则项后，导数计算中添加了惩罚因子 λ ，计算过程如公式 4-14 所示：

$$\frac{\delta J_{L2}(w)}{\delta w} \Big|_{w=0} = d + w * \lambda * w = d \quad (4-14)$$

可以看出，原损失函数加上 L2 正则化之后，在 0 处的导数不变，损失函数无影响。并且加入惩罚项之后，可以对模型的参数权重进行正则惩罚。对特征值求极限，沿着特征值较大的方向正则化影响较小，反之会收缩到几乎为零，也就是说这种不重要方向对应的分量会在训练过程中因正则化而逐渐衰减，以达到对训练过程中的参数权重进行惩罚的效果。

此外，在深度学习模型的设计和训练中，随着网络层数的增加，通常在反向传播过程中，由于多层网络结构导致梯度值在传播过程中迅速减小或增大，从而使得网络

深层的权重难以更新，很容易发生梯度消失 (Gradient Vanishing) 和梯度爆炸 (Gradient Explosion) 的问题，影响了模型的学习效率和性能。

为了避免梯度问题的发生,引入基于残差网络 (Resnet) 的残差单元 (Res-Block)。残差单元可以通过加权求和的方式，将输入样本直接映射到输出，同时保留经过一系列层变换后的特征。这种设计不仅简化了参数传递过程,而且通过跳跃式连接 (skip connections) 使得梯度能够更有效地在网络中流动，从而缓解了梯度消失和梯度爆炸的问题。在推理过程中，残差单元的加权方法减少了冗余特征的计算，提高了模型的计算效率。此外，残差模块能够学习到参数传递过程中的恒等映射，它在一定程度上还有助于减轻模型的过拟合问题。

如图 4.4 所示，基于残差网络模型的残差方法对图像特征的利用，使得音频数据样本的特征权重更加全面，能够依靠较少的参数达到较好的泛化性能，使得音频特征和反向梯度的传递更加高效。

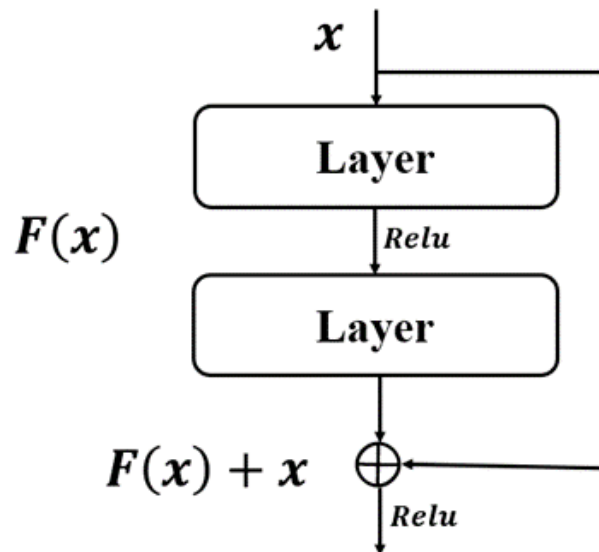


图 4.4 残差单元效果展示

4.3 训练策略

4.3.1 学习率递减策略

在模型的训练过程中，学习率的调整对于模型性能的提高至关重要。当模型在训练集上的表现逐渐接近全局最优时，即验证集损失值趋近于训练集损失值的全局最小值，这通常意味着模型已经学习到了一些有用的特征表示，但同时也可能意味着模型开始接

近某个局部最优解，而不再是全局最优解。

为了避免模型陷入局部最优，同时确保模型权重能够更细致地探索参数空间以接近拟合鞍点，引入学习率调整策略，即根据训练的进展动态地调整学习率的大小。这种策略使得在训练过程中，模型权重的更新变得更加精细，以避免错过更优的解。

学习率递减公式如公式 4-15 所示：

$$lr_{reduced} = factor * lr_{current} \quad (4-15)$$

其中， $lr_{current}$ 是当前的学习率， $factor$ 是学习率缩减比率，它决定了学习率降低的幅度。通过将当前学习率乘以这个减小比率，得到了衰减后的学习率 $lr_{reduced}$ ，它将用于接下来的训练步骤中。

这种引入学习率递减策略的可以允许模型在训练初期以较大的步长快速收敛，而在训练后期则减小步长，进行更细致的搜索，以期找到全局最优解或更好的局部最优解。此外，平滑递减学习率还可以帮助减少训练过程中的噪声，提高模型的泛化能力，从而在未知数据上也能有较好的表现。

4.3.2 梯度调整策略

在 4.2.3 小节中介绍了正则化和残差单元以防止模型的梯度出现消失和爆炸的问题，此外，在模型训练过程中，引入了梯度置零 (Zero Grad) 和梯度累加 (Accumulation) 方法来让模型的梯度更加稳定，同时也能起到加速模型训练的效果。

在模型的推理过程中，需要根据损失函数对模型的梯度进行下降操作，但每一轮的梯度计算会让模型变得十分繁琐。因此，根据深度学习框架的特性，本文引入了梯度置零方法，即在每一轮训练过程中，在模型根据梯度的变化更新网络参数之后，将梯度置零，从而加速下一轮模型的训练。

此外，这样的方法能够更自由的设计梯度，引入梯度累加策略，在每一轮训练过程中设置一个循环阈值 n ，在循环过程中不断累加模型推理所得到的梯度值，再根据 n 次循环得到梯度，更新网络模型的参数，更新梯度后再将梯度置零。如图 4.5 所示，在模型计算损失 (CalLoss)、前向传播 (Forward) 和后向传播 (Backward) 之后，添加一个循环累加单元，根据循环阈值循环 n 次，将 n 次循环的梯度进行累加，进而根据累加得到的梯度更新网络参数，并且后续加入梯度置零以不影响之后的训练的效果。这样的结合操作变相的扩大了模型的抓取尺度 (Batchsize)，能够在有效的保证模型的梯度不消失的情

况下，加速模型的推理，且不会影响到后续训练的梯度。

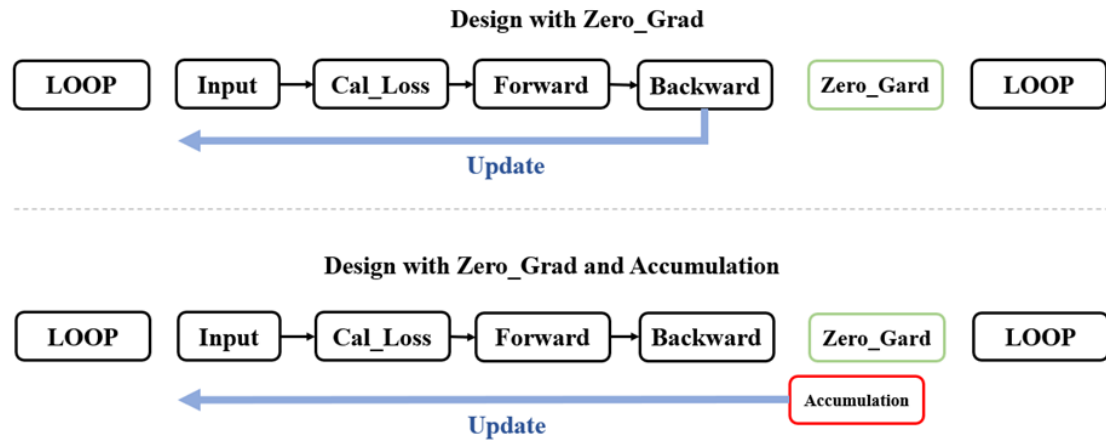


图 4.5 梯度置零和梯度累加

4.4 模型创新点

在搭建基于 GAN 和 FLOW 模型的仿声模型过程中，为了提高模型的整体性能和适应能力，经过了不断的努力尝试和创新，模型的创新点从图 4.6 所示的四个方面进行介绍。

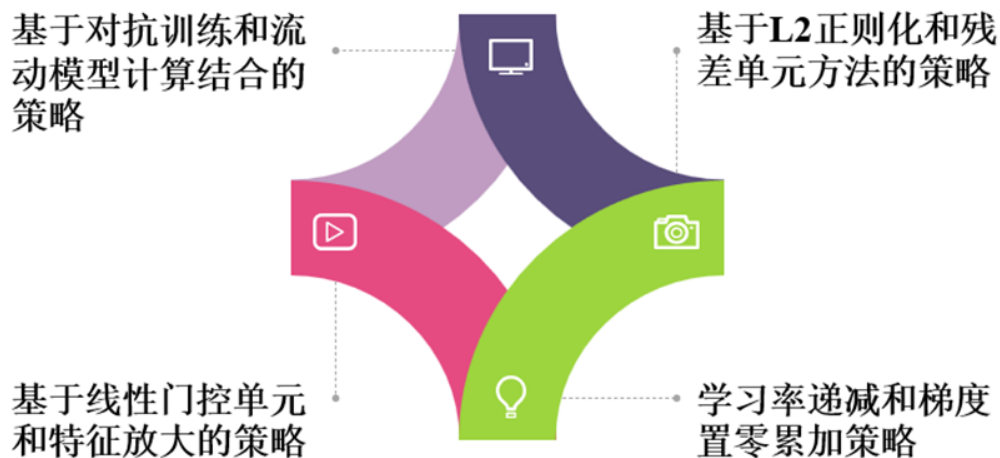


图 4.6 创新点概要

(1) 基于对抗训练和流动模型计算结合的策略

模型的设计基于对抗训练和流动模型计算相结合的策略。在模型的整体构建上，采用的基于对抗生成网络中的生成器和判别器的对抗训练的架构，这样的对抗训练可以有

效的减少模型推理过程中的参数计算量，较好的防止了训练过程中的特征冗余。同时，为了避免传统对抗训练过程中的近似计算导致的推理速度下降，采用流动生成模型的计算方法代替了对抗生成网络中的近似计算，这样在避免特征冗余的情况下，有效的加速了模型的计算过程，提升了模型的训练速度。

(2) 基于 L2 正则化和残差单元方法的策略

随着训练轮数的增加，特征往往会产生冗余现象，从而导致核心特征不显著，模型效果不佳的情况。本文采用正则化和残差单元的思想，首先在每一层网络中对参数进行惩罚，使得网络推理过程中的参数量得到了减少，其次加入了残差单元，根据残差单元的参数加权和跳跃式传递的方法，使得参数在推理过程中与参数本身的直接映射结果进行加权，使得参数量得到减少，此外跳跃式传递可以让参数跳过一些网络层次，达到减轻网络推理负担的效果。

(3) 基于线性门控单元和特征放大的策略

为了让模型更加侧重于核心特征的提取和推理，引入了线性门控单元和特征放大策略。在模型的上采样过程中，卷积的操作使得模型的感受野缩小，本文采用了线性门控单元来保证在采样过程中时序的稳定性，并扩大深层次的特征。此外，特征放大模块可以将卷积窗口内的数据进行放大，有利于模型对于主要特征的提取和训练。

(4) 学习率递减和梯度置零累加策略

在模型训练过程中，为了处理模型陷入局部最优解的问题和梯度不稳定的问题，本文加入了学习率递减策略和梯度置零累加策略。通过对学习率的阶段性调整，可以帮助模型跳出局部最优解，以获得更好的训练效果。利用梯度置零和累加策略，可以避免模型的梯度消失现象，一定程度上保证了模型梯度的稳定性，同时经过梯度累加策略，间接地扩大了模型的抓取尺度，加速了模型的推理速度。

5 评价指标和实验结果分析

5.1 评价指标

在传统的深度学习任务中，例如分类任务、预测任务和识别任务等，通常以精确率作为重要的指标。与传统的深度学习任务不同，对于音频的音色转换任务往往很难计算出音色转换的精确率。在此任务中，评价指标主要分为两大类，即主观指标和客观指标。

5.1.1 主观指标

在音频处理任务中，主观指标是十分重要的，它能够反应经由模型转换后音频与原音频听感上的相似度。不同于传统的深度学习任务中的对错分明，音色的转换很难有对错之分，所以在此任务中产生了主观指标。主观指标是指通过人的听感，去感受转换前后音频听感上的相似度并进行打分，通常用于模型测试阶段的自我评估。

5.1.2 客观指标

在客观指标中，选取模型的拟合速度和梅尔倒谱系数^[21]作为评价模型的性能和效果的指标。模型的推理速度在模型训练过程中十分重要，不管是在小批次数据集还是在工业级大批次数据集的训练中，推理速度都是至关重要的。梅尔倒谱系数 (Mel Frequency Cepstral Coefficients, MFCC) 是一种识别音频数据中人声特征的重要系数，它可以起到识别不同人声的作用。如图 5.1 所示，在得到音频数据的梅尔频谱 (Mel- Spectrum) 后，通过对数操作可以计算出音频数据的梅尔倒谱系数。

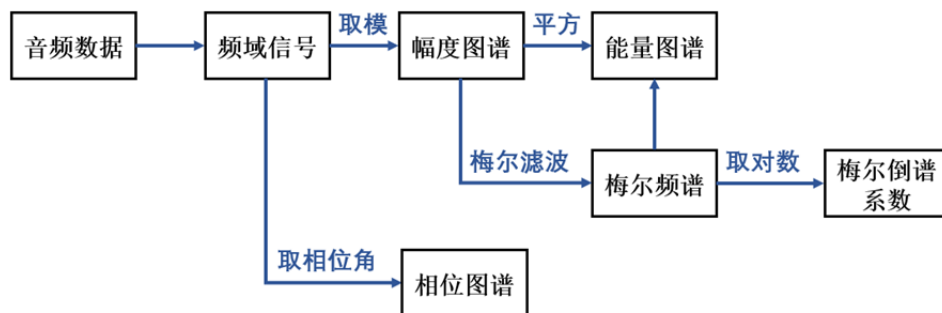


图 5.1 梅尔倒谱系数计算过程展示

计算出转换前后音频的梅尔倒谱系数后，利用转换后音频的梅尔倒谱系数减去原音

频梅尔倒谱系数，再与原音频梅尔倒谱系数进行比值计算，可以计算出转换前后音频的音色差距，如公式 5-1 所示。

$$MFCC_{Gap} = \frac{|MFCC_{Converted} - MFCC_{Origin}|}{MFCC_{Origin}} \quad (5-1)$$

其中 $MFCC_{Gap}$ 表示梅尔倒谱系数差距 (MFCC Gap), $MFCC_{Converted}$ 表示转换完成后的音频的梅尔倒谱系数, $MFCC_{Origin}$ 表示原数据的梅尔倒谱系数。梅尔倒谱系数差距越小, 表示模型的转换效果越好。

5.2 实验结果分析

5.2.1 实验环境设置

在本次实验中, 实验平台的硬件具体配置为: 16vCPU Intel(R) Xeon(R) Platinum 8352V 核中央处理器 (2.10GHz, 120GB 运行内存)、NVIDIA RTX 4090 (24G 显存) 图形处理器; 实验运行环境为: Windows11 64 位操作系统、Python-3.11 编程语言、librosa-0.9.1 音频处理库和 Pytorch-2.2 深度学习框架, 在实验中所有工具库都是最新版本。

5.2.2 模型评价

在模型的训练中, 总计训练 20000 轮, 其中生成器的损失 (Generator Loss, GL) 从 22.31 下降到 3.88, 判别器的损失 (Discriminator Loss, DL) 从 1.24 下降到 0.35, 设置每 5000 轮进行一次记录和模型的断点保存, 下降过程如表 5.1 所示。其中每次记录间隔时间记为 Time, 单位为秒 (s)。

表 5.1 模型训练过程展示

Epoch/Iter	0	5000	10000	15000	20000
GL	22.31	5.65(-16.66)	4.29(-1.36)	4.19(-0.1)	3.88(-0.21)
DL	1.24	0.43(-0.81)	0.39(-0.04)	0.37(-0.02)	0.35(-0.02)
Time/s	0	6082	6441	6387	6388

从表中可以看到, 随着训练的进行, 生成器和判别器的损失都得到了降低, 判别器最终损失降低到了 0.35, 说明生成器在训练中不断优化生成策略, 最终生成的数据样本

基本可以被判别器判断为原始特征，达到了音色转换的效果。此外，在模型推理速度上，迭代过程中平均每一代只需要 3-4 秒的时间，这极大的节省了模型的训练时间。在时间缩短的同时，也达到了生成器和判别器损失不断下降的效果。这一速度的提升显著减少了模型训练的整体时间消耗，使得训练过程更加高效。这种时间上的节省并没有以牺牲模型性能为代价，反而伴随着生成器和判别器损失的持续下降，这进一步证明了模型结构和训练策略的有效性。

同时，为了测试模型音色转换的效果，对转换前后的音频数据进行了梅尔倒谱系数计算，并根据它们的梅尔倒谱系数计算出了转换前后数据的梅尔倒谱系数差距。在测试数据集中，随机选取了 5 个测试样本进行测试，并使用不同训练轮数的模型进行音色转换测试，测试结果如表 5.2 和图 5.2 所示。其中，原始数据的梅尔倒谱系数记为 Ori-MFCC，通过模型转换后的音频梅尔倒谱系数为 Cov-MFCC。转换前后的音频数据的梅尔倒谱系数差距记为 $MFCC_{Gap}$ 。

表 5.2 测试结果展示

	Ori-MFCC	Cnv-MFCC1	Cov-MFCC2	$MFCC_{Gap}$
B-032.wav	-9.106	-11.117(-2)	-10.815(-1)	18%
B-090.wav	-6.298	-9.181(-3)	-6.870(-1)	9%
B-103.wav	-7.043	-9.626(-2)	-7.978(-1)	13%
B-115.wav	-7.621	-9.337(-2)	-7.994(-0)	4%
B-096.wav	-7.916	-9.864(-2)	-8.792(-2)	11%

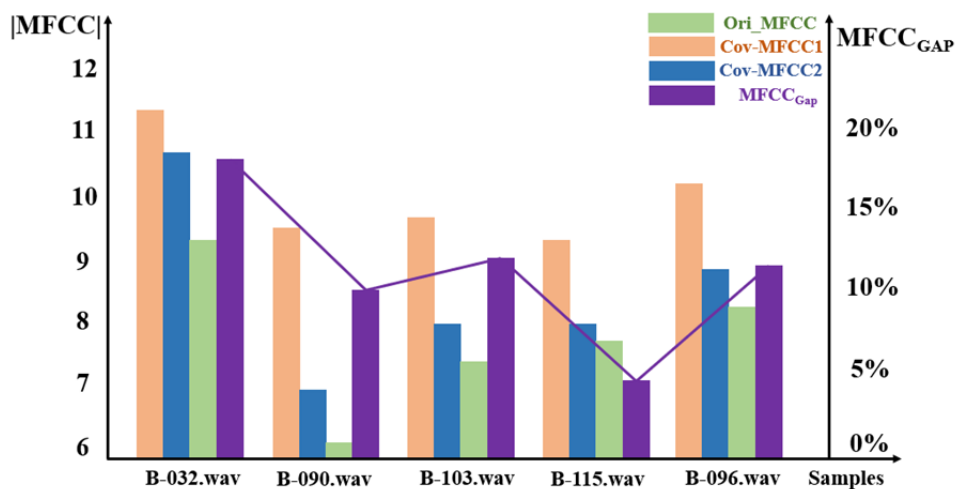


图 5.2 测试结果展示

可以看到，随着模型的推理训练，转换后的音频和原始音频的梅尔倒谱系数的差距明显减小，并且差距平均为 10% 左右，说明模型对于音色的转换性能优良，并且在主观听感上，音色也十分相近。

通过观察转换前后的梅尔频谱图的变化，可以看到转换后的音频数据在低分贝领域出现了原音频数据不具备的特征，如图 5.3 和图 5.4 所示，这是因为音频数据存在背景噪音。在音色转换过程中，模型将原音频的背景噪声也进行了训练，使得在音色转换时，将噪音也进行的转化，所以原音频数据的梅尔频谱图中表示的音色特征没有低分贝特征，在转换完后产生了低分贝音色特征。这说明计算所得的梅尔倒谱系数差距比实际要大，也间接说明了模型具有较强的转换能力。选用更高质量的音频数据进行训练可以有效的减少噪声特征的干扰，以达到更好的训练效果。

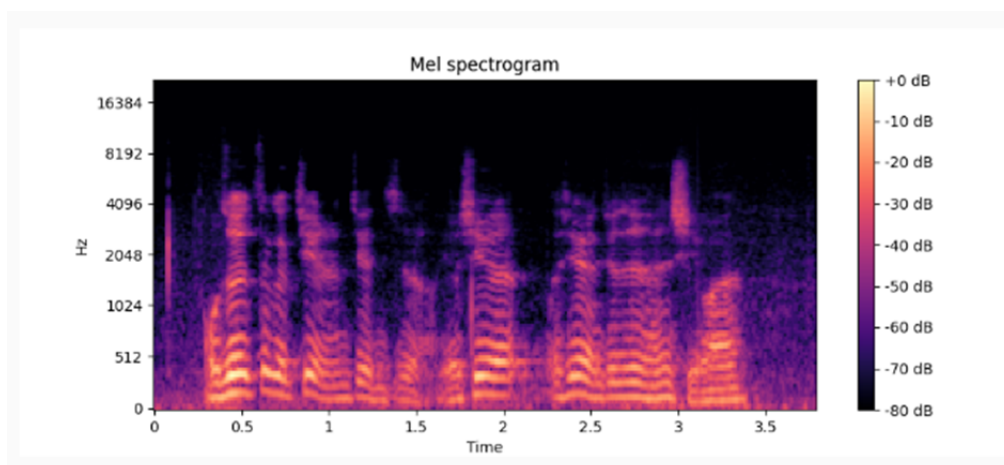


图 5.3 原音频数据的梅尔频谱特征可视化

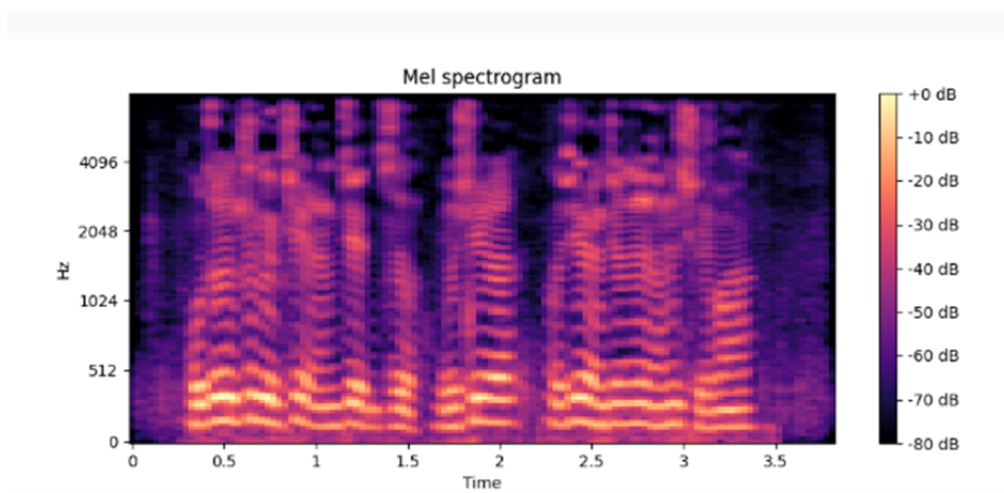


图 5.4 音色转换后的音频数据的梅尔频谱特征可视化

此外，将本次实验中训练的模型 myGAN 与其他相关模型进行了速度和性能上的比较，结果如表 5.3 和图 5.5 所示。

表 5.3 花费时间对比

Epoch/Iter	0	5000	10000
myGAN	0	6082	6441
CycleGAN	0	6337	6471
StarGAN	0	6402	6549
HiFiGAN	0	6051	6585
VITS	0	6865	6817

从表 5.3 中可以看出，通过 10000 轮的对比试验，实验模型 myGAN 的平均每一轮的时间花费为 1.3 秒，推理速度要快于其他相关模型。其中 HiFiGAN 和 VITS 模型是基于 TTS(Text to Speech) 的音色转换模型，需要音频和文字的数据对传入进行训练，推理时参数计算量有所扩大，因此推理速度通常会慢于基于 STS(Speech to Speech) 的模型。

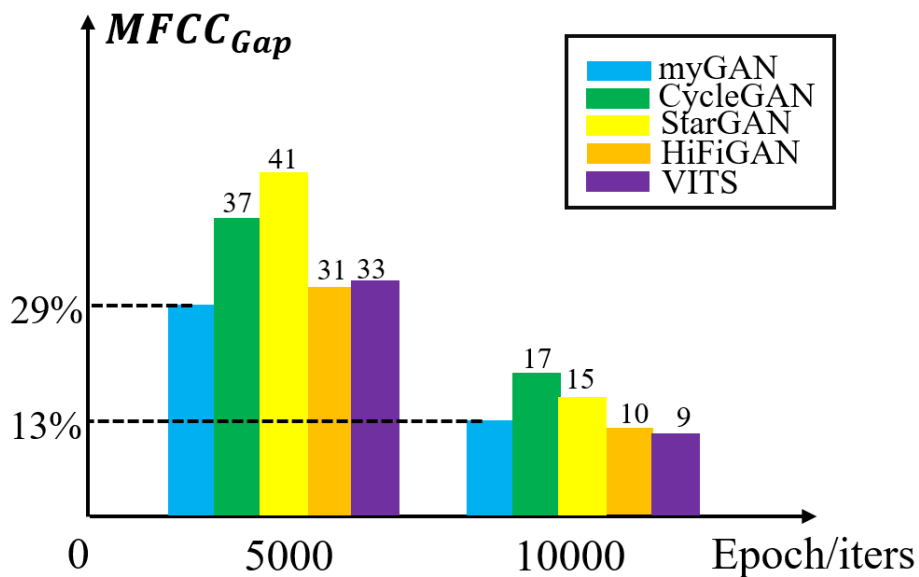


图 5.5 梅尔倒谱系数差距对比

从图 5.5 中可以看出，实验模型 myGAN 在训练前期，拟合速度很快，这证实了模型策略的有效性。但是在高轮次训练中，随着其他相关模型的逐渐拟合，模型的最终性能会略差于基于 TTS 的 HiFiGAN 和 VITS。因此，实验模型在具备了较好的性能的基础上，速度有着很大的优势。

6 Web 网站前后端设计

设计 Web 前后端系统采用了基于 Vue3 的 Quasar 前端框架和基于 Python 的 Fastapi 后端框架。通过 Vue3 语法在前端设计可视化界面和按钮，向后端发送请求，并通过 axios 传递参数，以完成目标功能。

6.1 系统主页面

启动前端和后端服务，访问本地 9000 端口来查看网页设计的主页，如图 6.1 所示。在主页面中包括了以下模块：文件上传模块；音频数据特征展示模块；音频特征可视化模块；音色转换模块。



图 6.1 系统主页面展示

在系统主页面的设计中，使用了 Quasar 框架中的 q-banner 标签设计了 Web 页面的导航栏，使用了 q-file 标签用于启动本地文件选择器，使用 q-btn 标签设计了按钮来响应函数，使用了 audio 标签来对缓存到前端的音频文件进行预览和播放，使用 q-card 标签设计了音频文件特征可视化展示模块。Web 页面设计代码结构如图 6.2 所示。

```

<template>
  <!-- 导航栏部分 -->
  <q-banner class="">...
</q-banner>
  <!-- 分割线 -->
  <q-separator color="blue-grey-2" size="2px" inset />
  <!-- 文件选择器&&音色转换模块 -->
  <div class="q-gutter-md q-pl-xl q-pt-md float-left" style="max-width: 21%">...
</div>
  <!-- 音频数据可视化 -->
  <div class="q-gutter-md q-pl-xl q-pt-md float-left" style="max-width: 80%">...
</div>
  <!-- 图片数据传递模块 -->
  <div class="q-gutter-md q-pl-xl q-pt-md float-left" style="max-width: 80%">...
</div>
  <!-- ----- -->
</template>

```

图 6.2 系统主页面代码结构

6.2 文件上传模块

在主页面中，如图 6.3 所示，通过点击表单按钮，可以调用文件选择器，从而选择需要进行音色转换的 wav 音频文件。

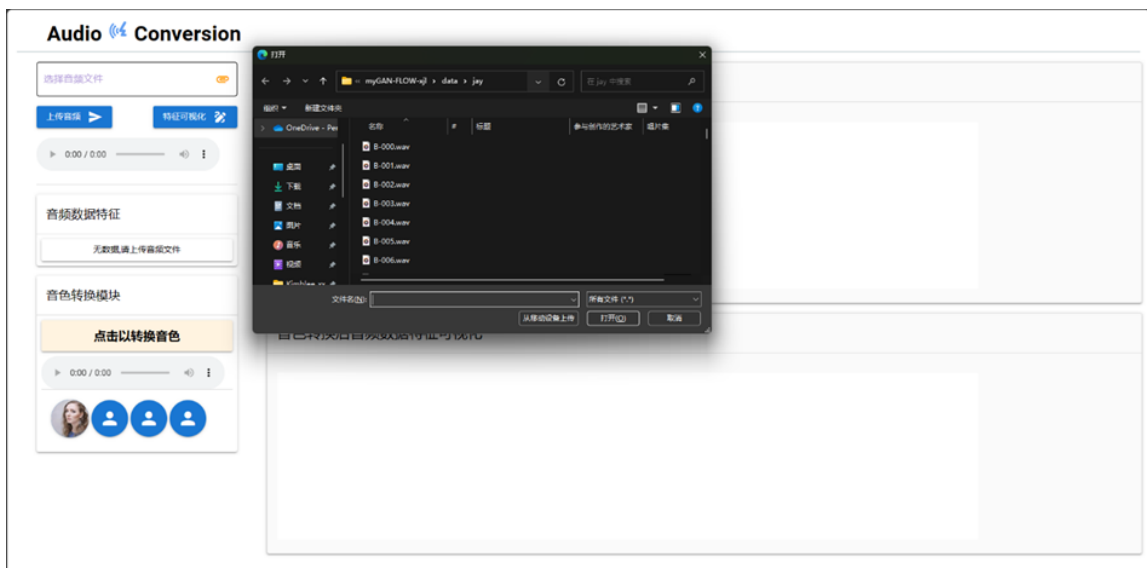


图 6.3 音频文件上传展示

在设计文件上传模块时，使用 q-file 标签启动文件选择器，以从本地上传音频文件，并使用了 use-chips 属性，使得选择完文件后在前端页面的表单中显示文件名称。此外，绑定了 handleFileChange 函数以响应文件上传，以便于后续的前后端数据传输操作。文件上传模块的代码设计如图 6.4 所示。

```
<q-file
  color="black" outlined label-color="deep-purple-3" use-chips
  v-model="audioFile" label="选择音频文件" @change="handleFileChange"
  style="max-width: 100%"
>
```

图 6.4 文件上传模块代码实现

选择音频文件后，可以在线播放来预览声音内容，同时可以调节上传音频文件的播放进度、音量大小和播放速度，如图 6.5 所示。



图 6.5 对上传音频的预览操作展示

在音色转换这一任务中，前端页面中预览播放音频数据至关重要，它可以让用户直观听到上传的音频文件的内容和音色。在代码设计上，使用 `audio` 标签进行监听文件的上传，结合“上传音频”按钮绑定的 `handleFileChange` 函数，可以做到音频上传后即可播放的效果。代码实现如图 6.6 所示。

```
<audio ref="oriaudioPlayer" controls>
  <source :src="oriaudioUrl" type="audio/wav" />
  Your browser does not support the audio element.
</audio>
```

图 6.6 前端预览播放功能代码实现

6.3 音频数据特征展示模块

通过点击“上传音频”按钮，可以将音频文件传递给后端系统，后端给出响应，通过 `wavefile` 和 `librosa` 库计算出音频数据的特征，以 `Json` 格式传递音频数据的音频时长、

音频采样率、音频大小、音频比特率和音频的声道数给前端页面，前端页面通过 axios 接收后在页面中展示出来。如图 6.7 所示。



图 6.7 音频数据特征展示

在音频数据特征展示模块中，利用了 axios 库协助前后端音频数据的传输。在前端页面中，将音频数据以流的形式传输给 Fastapi 后端，后端接收文件后，整合出文件的地址，并通过音频处理库对音频数据加以计算，得出音频数据的各项特征，并以 JSON 的格式返回给前端。在前端的设计中，使用 v-if 语句来监听后端是否成功返回数据特征，如果成功返回则在页面中展示。后端设计代码如图 6.8 所示。

```
def analyze_audio(file_path):  
    with wave.open(file_path, 'rb') as wf:  
        # 获取音频基本信息  
        params = wf.getparams()  
        sample_width = wf.getsampwidth()  
        channels = wf.getnchannels()  
        sample_rate = wf.getframerate()  
        duration = params.nframes / params.framerate  
        file_size = os.path.getsize(file_path)  
        # 计算比特率  
        bitrate = sample_rate * sample_width * 8 / 1000  
        # 读取音频数据  
        data = np.frombuffer(wf.readframes(params.nframes), dtype=np.int16)  
    return {  
        "duration": duration,  
        "sample_rate": sample_rate,  
        "file_size": file_size,  
        "bitrate": bitrate,  
        "channels": channels,  
        "encoding_format": "PCM",  
    }
```

图 6.8 后端计算数据特征功能代码实现

6.4 音频特征可视化模块

通过点击“特征可视化”按钮，可以将上传音频的波形图和梅尔频谱图显示出来。梅尔频谱特征是音色转换的重要核心特征，在前端页面中展示可以很直观的看到转换前后音频文件特征的差异。前端给后端发送请求，后端通过 librosa 和 matplotlib 库绘制出音频文件的波形图和梅尔频谱图，以 buffer 流形式传递给前端页面，前端页面通过 axios 接收后在页面中显示出来。如图 6.9 所示。音色转换后的音频文件的可视化如图 6.12 所示。

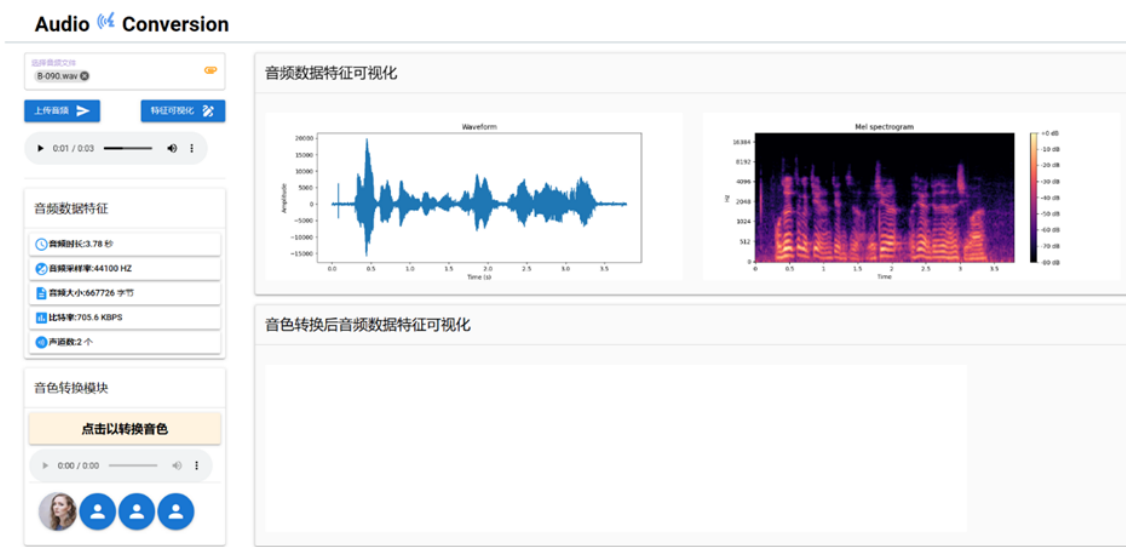


图 6.9 音频数据特征可视化展示

音频数据特征可视化功能的代码设计中，PNG 格式的特征可视化图以 Buffer 流的形式在前后端之间传输。在 Web 页面中使用 q-card 标签设计了可视化展示框，使用 v-if 来监听后端是否成功的将可视化图传递到前端，传递成功后在页面中展示出来。在后端使用第三方库音频处理库和图形绘制库来对前端上传的音频文件进行特征可视化。后端波形图和梅尔频谱图的绘制功能的实现代码如图 6.10 和 6.11 所示。

```
def analyze_wave_pic(file_path):
    with wave.open(file_path, 'rb') as wf:
        count=100
        # 获取音频基本信息
        params = wf.getparams()
        duration = params.nframes / params.framerate
        # 读取音频数据
        data = np.frombuffer(wf.readframes(params.nframes), dtype=np.int16)
        # 绘制波形图
        plt.figure(figsize=(10, 4))
        plt.plot(np.linspace(0, duration, len(data)), data)
        plt.xlabel('Time (s)')
        plt.ylabel('Amplitude')
        plt.title('Waveform')
        buffer = BytesIO()
        plt.savefig(buffer, format='png')
        plt.savefig("./pic_wave/wave_form_{}.png".format(count))
        count -= 1
        plt.close()
        waveform_img = base64.b64encode(buffer.getvalue()).decode('utf-8')
    return waveform_img
```

图 6.10 后端绘制音频数据波形图功能的代码实现

```
def analyze_mel_spectrogram(file_path):
    count = 100
    y, sr = librosa.load(file_path, sr=None)
    S = librosa.feature.melspectrogram(y=y, sr=sr)
    plt.figure(figsize=(10, 4))
    librosa.display.specshow(librosa.power_to_db(S, ref=np.max), sr=sr, x_axis='time', y_axis='mel')
    plt.colorbar(format='%+2.0f dB')
    plt.title('Mel spectrogram')
    buffer = BytesIO()
    plt.savefig(buffer, format='png')
    plt.savefig("./pic/mel_spectrogram_{}.png".format(count))
    count -= 1
    plt.close()
    mel_spectrogram_img = base64.b64encode(buffer.getvalue()).decode('utf-8')
    return mel_spectrogram_img
```

图 6.11 后端绘制音频数据梅尔频谱图功能的代码实现

6.5 音色转换模块

通过点击左侧下方的圆形任务按钮，可以预览播放目标音色。点击“点击以转换音色”按钮，即可对上传音频进行音色转换操作。在音色转换完成后，可以在 Web 页面预览播放转换后的音频，并可以下载到本地文件夹中。此外，系统可以自动的展示出转换后音频的波形图和梅尔频谱图，前端页面中上下展示以对比转换前后的特征变化和相似度。如图 6.12 所示。

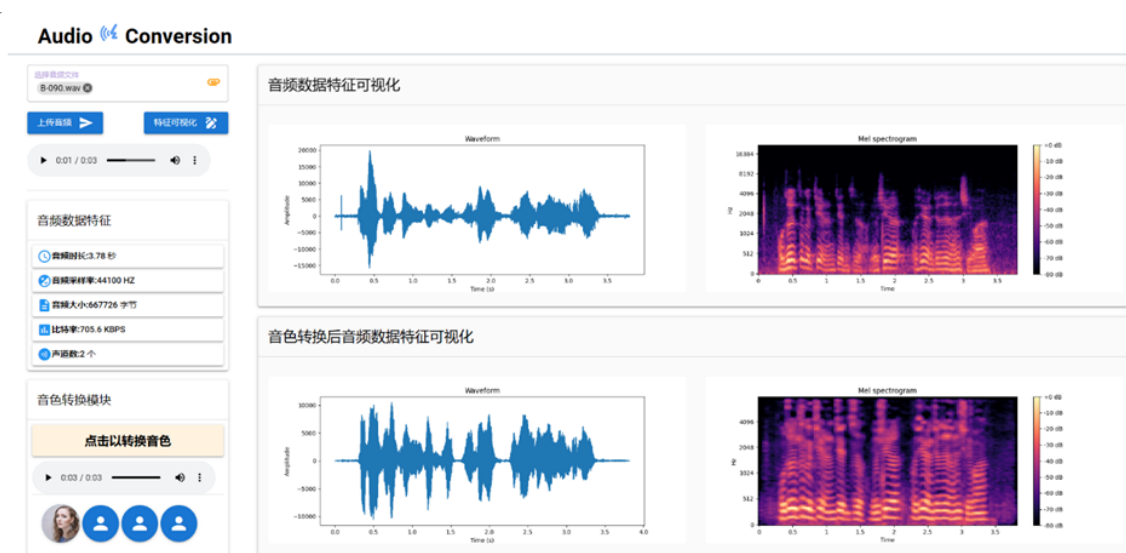


图 6.12 音色转换和特征可视化对比展示

将前端上传的音色数据进行音色转换操作需要调用本文算法部分训练好的深度学习模型，在后端设计部分，将前端上传的文件传入到模型测试模块中，并使用 Pytorch 深度学习框架展开模型参数推理，将转换后的音频文件输出，并以 Stream 流的形式返回给前端页面。在前端页面的设计中，使用 audio 标签来接收后端返回的音频数据，

以实现前端预览播放的功能。此外，设计了音色转换后音频数据的特征可视化模块以方便地与原音频数据的可视化特征进行对比，能够更加直观地看到音频数据音色特征的变化以及变换后的特征差异。核心代码实现如图 6.13 所示。

```
def audioConv(file_path, output_path):
    parser = argparse.ArgumentParser(
        description="test myGAN")
    logf0s_normalization_default = './cache/logf0s_normalization.npz'
    mcep_normalization_default = './cache/mcep_normalization.npz'
    coded_sps_A_norm = './cache/coded_sps_A_norm.pickle'
    coded_sps_B_norm = './cache/coded_sps_B_norm.pickle'
    model_checkpoint = './model_checkpoint/'
    resume_training_at = './model_checkpoint/_myGAN_CheckPoint'
    argv = parser.parse_known_args()[0]
    logf0s_normalization = argv.logf0s_normalization
    mcep_normalization = argv.mcep_normalization
    coded_sps_A_norm = argv.coded_sps_A_norm
    coded_sps_B_norm = argv.coded_sps_B_norm
    model_checkpoint = argv.model_checkpoint
    resume_training_at = argv.resume_training_at
    validation_A_dir = argv.validation_A_dir
    output_A_dir = argv.output_A_dir
    validation_B_dir = argv.validation_B_dir
    output_B_dir = argv.output_B_dir
    if not os.path.exists(logf0s_normalization) or not os.path.exists(mcep_normalization):
        print(
            "Cached files do not exist, please run the program preprocess_training.py first")
    myGAN = myGANTest(logf0s_normalization=logf0s_normalization,
                      mcep_normalization=mcep_normalization,
                      coded_sps_A_norm=coded_sps_A_norm,
                      coded_sps_B_norm=coded_sps_B_norm,
                      model_checkpoint=model_checkpoint,
                      validation_A_dir=validation_A_dir,
                      output_A_dir=output_A_dir,
                      validation_B_dir=validation_B_dir,
                      output_B_dir=output_B_dir,
                      restart_training_at=resume_training_at)
    myGAN.test()
```

图 6.13 后端音色转换功能的代码实现

通过对音色转换前后音频数据特征的对比，可以看到在低分贝领域，转换后的音频的梅尔频谱特征比原音频的特征更为明显，这是因为在模型训练过程中，数据集的背景音的存在着差别，说明在转换过程中，将原音频的背景音一并转换了，这就使得转换后的音频在低分贝区域内具有了一些特征。但是这些特征处于低分贝区域，人耳很难判别出这种声音，对模型的影响相对较小。通过使用更高质量的数据集可以有效的避免这种现象的发生。

7 总结

针对音色转换任务，本文设计了一种基于 GAN 和 FLOW 模型的仿声系统来对音频数据进行音色的转换，并引入了线性门控单元、特征放大策略、L2 正则技术、残差单元方法以及流动生成计算方法，使得模型的推理更加迅速，在保证模型音色转换效果的同时，缩短了模型拟合所需要的时间。此外，本文还设计了基于 Vue3 的 Quasar 前端框架的前端 Web 页面和基于 Python 的 Fastapi 框架的后端服务，实现了在 Web 页面中对音频数据的预览和音色转换功能，使得用户可以从本地上传音频数据进行音色转换操作，并可以直观感受到音色的变化，结合数据可视化等技术，前后端系统直观地展示了音色转换前后数据特征的差异。但是，针对不同语种的音色转换任务，模型并未体现出很好的效果，因此模型在不同语种的音色转换效果上仍有上升空间。

参考文献

- [1] Kim J, Kong J, Son J. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech[C]. In *Proceedings of the 38th International Conference on Machine Learning*. July 18-24, Virtual, 2021, pp. 5530-5540.
- [2] Kingma D P, Welling M. Auto-encoding variational bayes[J]. arXiv preprint arXiv:1312.6114, 2013.
- [3] Ping W, Peng K N, Zhao K X, et al. Waveflow: A compact flow-based model for raw audio[C]. In *Proceedings of the 37th International Conference on Machine Learning*. February 07-12, New York, USA, 2020, pp. 7706-7716.
- [4] Hao Y Y, Chen Y B, Zhang W W, et al. A real-time music detection method based on convolutional neural network using Mel-spectrogram and spectral flux[C]. In *Proceedings of the 51th INTER-NOISE and NOISE-CON Congress and Conference*. August 01-05, Washington, D.C., USA, 2021, pp. 5910-5918.
- [5] Sheinin M, Chan D, O'Toole M, et al. Dual-shutter optical vibration sensing[C]. In *Proceedings of the 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 18-24, Los Angeles, USA, 2022, pp. 16324-16333.
- [6] Li T, Hu C X, Cong J, et al. DiCLET-TTS: Diffusion model based cross-lingual emotion transfer for text-to-speech—A study between English and Mandarin[J]. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023, 31: 3418-3430.
- [7] Yang L, Zhang Z L, Song Y, et al. Diffusion models: A comprehensive survey of methods and applications [J]. *ACM Computing Surveys*, 2023, 56(4): 1-39.
- [8] Dauphin Y N, Fan A, Auli M, et al. Language modeling with gated convolutional networks[C]. In *Proceedings of the 34th International conference on machine learning*. August 06-11, Sydney, Australia, 2017, pp. 933-941.
- [9] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial networks[J]. *Communications of the ACM*, 2020, 63(11): 139-144.
- [10] Andrieu C, De Freitas N, Doucet A, et al. An introduction to MCMC for machine learning[J]. *Machine learning*, 2003, 50(1): 5-43.
- [11] Kaneko T, Kameoka H, Tanaka K, et al. Cyclegan-vc3: Examining and improving cyclegan-vc3 for mel-spectrogram conversion[J]. arXiv preprint arXiv:2010.11672, 2020.
- [12] Choi Y, Choi M, Kim M, et al. Stargan: Unified generative adversarial networks for multi-domain image-to-image translation[C]. In *Proceedings of the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 18-22, State of Utah, USA, 2018, pp. 8789-8797.
- [13] Lei Y, Yang S, Cong J, et al. Glow-WaveGAN 2: high-quality zero-shot text-to-speech synthesis and any-to-any voice conversion[J]. arXiv preprint arXiv:2207.01832, 2022.
- [14] Binkowski M, Marti G, Donnat P. Autoregressive convolutional neural networks for asynchronous time series[C]. In *Proceedings of the 35th International Conference on Machine Learning*. July 10-15, Stockholm, Sweden, 2018, pp. 580-589.
- [15] Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models[J]. *Advances in neural information processing systems*, 2020, 33: 6840-6851.
- [16] He K M, Zhang X Y, Ren S Q, et al. Deep residual learning for image recognition[C]. In *Proceedings*

- of the 2016 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 27-30, Nevada, USA, 2016, pp. 770-778.
- [17] Ruder S. An overview of gradient descent optimization algorithms[J]. arXiv preprint arXiv:1609.04747, 2016.
- [18] Shi W Z, Caballero J, Huszár F, et al. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network[C]. In *Proceedings of the 2016 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 27-30, Nevada, USA, 2016, pp. 1874-1883.
- [19] Graves A. Long short-term memory[J]. *Supervised sequence labelling with recurrent neural networks*, 2012, 385: 37-45.
- [20] Schuster M, Paliwal K. Bidirectional recurrent neural networks[J]. *IEEE Transactions on Signal Processing*, 1997, 45(11): 2673-2681.
- [21] Abdul Z K, Al-Talabani A K. Mel Frequency Cepstral Coefficient and its Applications: A Review[J]. *IEEE Access*, 2022, 10: 122136-122158.

致 谢

行文至此，该为我的本科学习生活画上句号。在浙江财经大学四年的学习生活让我受益良多。我要向这四年里教导我知识、关心我生活的老师和同学们表示衷心的感谢。

首先，我要感谢我大学本科期间的导师张帅老师，张老师在这四年里教会了我最多的东西，让我在大学刚入学就努力学习 AI 领域的知识，为我四年的学习科研打下了坚实的理论基础。张老师教会了我撰写各种学术文档的技巧，吃苦耐劳的奋斗精神以及严谨细心的科研精神，这些宝贵的知识和精神财富让我受益良多。

接着我要感谢陈伟锋、张林、刘东威和陈明晶四位老师。陈伟锋老师全栈知识的教导、张林老师数据挖掘技术的指导、刘东威老师机器学习算法的讲解和陈明晶老师幽默风趣的 Python 教导我都会铭记在心。

然后我要感谢崔荣珍和杨秋逸同学，在生活中带给我乐趣，在学习中给予我鼓励和帮助。杨同学教会了我很多前后端设计的技巧，让我的前后端开发水平提升很多。同时也感谢崔同学在考研期间的鼓励和陪伴，让我在考研期间枯燥低落的生活也充满了乐趣。

最后我要感谢我的父母，是您们给了我这样一个机会能够来到这里学习和生活。在大学四年中，给予我物质支持和精神支持。您们对我的关心无微不至，总是会在我遇到困难或是心情低落时给我鼓励和帮助，让我重新乐观开朗的面对生活。

在最后这段时间里，匆匆忙忙，想往前看却又有很多不舍，希望这段经历能够让我未来的生活更加充实美好！